

Engineering Decisions Behind the Teams Agentic AI Platform

A walkthrough of how I designed, built, and shipped real-time conversational AI inside Microsoft Teams — and the calls I made (and a few I'd remake) along the way.

Raju Roopani · Senior Software Engineer / AI Tech Lead, Microsoft Teams

This isn't a list of frameworks. It's the story of one program I led from 0→1 to GA inside Microsoft Teams — what we were trying to do, the load-bearing decisions I made along the way, and the few I'd revisit if I were starting over. Where the decisions mattered I've tried to show the reasoning, not just the outcome.

1. What the system does, and why it matters

Microsoft Teams serves 300M+ daily active users, and meetings are the highest-bandwidth knowledge exchange in most enterprises. Before this program, that bandwidth was lossy: transcripts existed but were post-hoc artifacts, action items walked out the door with whoever happened to be writing them down, and most of the context of the meeting died with the meeting itself.

The system I led has two tightly-coupled pieces.

The Meetings AI layer. Real-time conversational AI that runs alongside a live meeting. It listens to the streaming transcript, retrieves relevant context (calendar, prior meetings, task systems, chat history), and offers in-meeting assistance — answering questions, drafting follow-ups, surfacing related decisions — with sub-200ms perceived latency. Under the hood it's a streaming LLM tool-use loop wrapped around a hybrid RAG pipeline reading a corpus that is literally growing as the meeting happens.

The Agent Extensibility Platform. A runtime, SDK, and set of APIs that lets internal and external developers build agents that live inside Teams — at-mentionable, multi-agent, with the right primitives for context propagation, turn-taking, human-in-the-loop intervention, and partial-failure isolation. It speaks A2A (agent-to-agent) and MCP (Model Context Protocol) as first-class. At GA it backed agents reaching the full 300M+ DAU surface.

Why it mattered. The difference between 'AI that helps you' and 'AI that lives where you already work' is enormous in enterprise. The latter compounds — your tools get smarter together — and Teams is the only surface where a meeting-aware AI can be born without forcing users to switch context.

2. My role

Tech lead and lead architect across both pieces. IC role, not management. Across the program I worked with ~12–15 engineers (plus a separate AI Research org) and was personally on the hook for:

- The orchestration runtime architecture: turn-taking protocol, context propagation, failure model, tenancy story.
- The Agent SDK surface: what does it feel like for a third-party developer to build an agent that behaves correctly under the runtime's constraints.

- The real-time RAG pipeline: what retrieval shape actually works over a corpus that doesn't exist when the meeting starts.
- The ML / Eng integration contract: the boundary between what AI Research owns (models, prompts, evals) and what Engineering owns (latency, reliability, rollout) — and the substrate that lets both sides ship independently.
- Binding architectural decisions where teams disagreed.

Concretely: I wrote the load-bearing design docs, prototyped the hardest pieces (runtime turn-taking, streaming RAG) myself before handing them off, reviewed PRs across the runtime, and spent a lot of time saying no to features so we could ship a thing that worked.

3. Where I reused what existed — and why

The fastest way to ship a bad AI system was to over-build. The default move every quarter was to reach for what already existed and spend our build budget on the things only we could build.

Foundation models — Azure OpenAI and Anthropic Claude, behind a thin internal interface. I never seriously entertained training our own. Model quality changes month-to-month; the moment you commit to one model in your architecture, you're paying for that decision for years. The abstraction layer was deliberately thin — just enough to swap providers, route by task complexity (tiered routing later cut cost per request 40%), and unify telemetry. It explicitly did *not* try to be a 'framework'. Those grow tentacles.

Vector retrieval — managed vector services on Azure. Hosting our own index in the early days would have been a multi-quarter distraction for marginal gain. Meeting traffic is bursty and dense; managed gave us cold-start in seconds. We revisited this only when retrieval latency on the long tail started showing up in evals.

Speech-to-text — Azure Speech Services. ASR is a domain of its own with a dedicated org. I had zero interest in building expertise we didn't need to own.

MCP and A2A as protocol standards. This was a build/buy decision in disguise. Teams is big enough that we could have invented our own agent-to-agent and tool-use protocols and forced adoption. I deliberately didn't. Third-party developers were already converging on MCP, and 'Teams works with the protocol you already speak' is a much stronger pitch than 'Teams has its own protocol.' Embracing the standard cost us some optimization headroom and gave us a far larger ecosystem on day one.

AutoGen for the earliest multi-agent prototypes. We used it to learn what multi-agent orchestration *should* feel like before committing to our own runtime. We discarded it before GA (see next section), but designing the runtime from a blank page without that grounding would have led us to a worse design. The cost of throwing away the prototype was much smaller than the cost of skipping the learning.

4. Where I built from scratch — and why

Four pieces, in roughly the order we admitted we had to build them.

The orchestration runtime. This was the call I argued hardest for. AutoGen, LangGraph, and the open frameworks at the time were single-process, single-tenant, and assumed batch-shaped workloads. We needed something distributed, multi-tenant, low-latency, and meeting-shaped: agents joining and leaving as participants do, turn-taking that respects who actually has the floor, idempotent retries because participants re-trigger the same intent under flaky networks, and partial-failure isolation so one

misbehaving 3rd-party agent couldn't take down the meeting. There was no honest 'buy' option. So we built one — event-driven, with a turn-taking protocol I wrote up as a small internal spec, agent identity and context propagation as first-class concepts, and a failure model that treated partial degradation as the normal case rather than the exception.

A real-time RAG pipeline over the live transcript. Off-the-shelf RAG assumes a static corpus. We needed retrieval over a corpus that grew as the meeting progressed, where 'relevance' depends on speaker, topic shifts, and time — a customer mentioned in minute 3 might be the thing the agent needs to retrieve about in minute 27. We built a streaming chunker that respected semantic boundaries (utterance + speaker + topic), a hybrid retrieval layer that combined embedding similarity with structured filters (calendar links, task IDs, participant identity), and a context-window manager that knew which retrieved items were still load-bearing vs. could be evicted. None of these pieces were exotic individually; the combination was specific enough to our shape that no existing framework fit cleanly.

The Agent SDK manifest schema. Existing SDKs were code-first — written for engineers comfortable instantiating an *Agent* class. Our 3rd-party population was much broader, including PMs and customer-org IT admins. The SDK had to be declarative: a manifest where you specify intent, retrieval surfaces, tool permissions, and HITL defaults, and the runtime fills in correct behavior. Code-first escape hatches sat underneath, but the declarative manifest was the headline. This is the piece I'm most proud of — when we measured it, 3rd-party agent reliability went up 3x and developer support tickets dropped 60%, almost entirely because the right defaults were locked into the manifest shape rather than left as opportunities for misuse.

The evaluation and observability harness. Model providers' tooling stops at the model. We needed full-loop eval — retrieval quality, end-to-end latency at p95, agent behavior under turn-taking pressure, regression detection when AI Research rolled new prompts or models, progressive rollout gates that could halt a rollout before regressions reached production. We built this. Partly because it didn't exist; mostly because owning it made the AI Research / Engineering contract concrete — AI Research could change anything they wanted *as long as it cleared our eval harness*, and Engineering could enforce that contract without needing to read prompts.

5. Tradeoffs I made — and lived with

Streaming UI vs. complete-response UI. Streaming tokens as the model produced them dropped perceived latency from ~3s to ~200ms. The cost was a UI that now had to handle partial states, retraction (the model occasionally correcting itself mid-stream), and graceful degradation when the stream dropped. We absorbed the complexity because in a *meeting* — where the AI is competing with humans speaking — feeling instant matters far more than feeling complete. I still think that was right.

Tiered model routing. Routing simple intents to a smaller model and reserving the frontier model for harder calls cut cost per request 40%. The cost was a routing classifier that itself could be wrong, and an integration test surface that grew with the number of tiers. The mitigation was the eval harness — every routing decision was scored offline before it could ship. If we hadn't built evals first, this would have been a bad call.

HITL on by default in the SDK. Every agent the SDK produced had a human-in-the-loop confirmation step by default for any action that wrote to a customer system. This made agents feel slower and chattier than they could have been if you trusted them. The tradeoff was conservative-by-default for enterprise customers who had no patience for AI taking actions on their behalf incorrectly. The escape hatch — explicit opt-out per action — was available, but you had to ask for it. I'd make the same call again.

Embracing MCP and A2A early. This cost us some optimization (an internal protocol could pack data more efficiently) and locked us into a moving target — the standards evolved under us. The win was ecosystem: external developers showed up speaking our protocols on day one. The cost showed up later, as compatibility work when the standards shifted. Net positive, but with eyes open.

Building our own runtime. The decision I'd defend most strongly and second-guess most often. The argument for: no off-the-shelf framework fit our shape, so we'd have ended up forking one anyway. The argument against: we now own and maintain a runtime forever. The honest answer is that I think it was correct at the time *and* I keep watching the open-source space carefully, because if a community-maintained runtime ever catches up to our needs, the right move is to migrate to it and stop carrying the maintenance load.

6. What I'd do differently in hindsight

Three things, in increasing order of 'I should have known better.'

The eval harness should have come first, not third. We shipped the orchestration runtime and the first agents to GA before we had a real evaluation harness in place. We caught a few embarrassing regressions in production that an eval suite would have flagged on commit. The classic mistake: eval feels like infrastructure investment, and infrastructure loses to features when you're trying to ship. I'd now treat the eval harness as a P0 deliverable in the same release as the first real model integration, not a fast-follow.

We over-built the multi-tenant story before we had one customer use case fully nailed. I had a real bias toward 'design for the platform we'll be in two years'. That meant the runtime carried complexity (tenant isolation primitives, per-tenant rate limits, multi-tenant configuration surfaces) before we had any tenant that needed it. If I were doing it again, I'd ship a single-tenant runtime first, get one use case completely solid, and only then generalize. We could have shipped two months earlier and learned more from real usage in those two months than we did from the extra abstraction.

I should have watched the open-source space longer before committing to our own runtime. The decision to build was right *at the moment we made it*. But I made it earlier than I had to. If I'd held off another quarter and watched what the community shipped, I might have been able to base our runtime on something rather than starting from a blank page. The kind of decision where being right too early is almost as expensive as being wrong.

A pattern across all three: *delay the irreversible decisions, accelerate the reversible ones*. I knew this in principle. Shipping under pressure made it harder to live by than I'd like to admit.

Closing note

The pieces of this program I'm proudest of aren't the technologies. They're the contracts: the eval harness as the boundary between AI Research and Engineering, the manifest as the boundary between the platform and 3rd-party developers, the turn-taking protocol as the boundary between agents. Each one took an ambiguous, negotiation-heavy interface and turned it into something a machine could check — and that's mostly what let us ship at scale without the program collapsing under coordination cost.

Happy to go deeper on any of this in conversation.